

Tabular Classification

LIN TONG, G2608825L

IN6227-Assignment-1

Variant-2

INTRODUCTION

This report presents an end-to-end tabular classification workflow applied to the dataset (31,112 rows, 15 features). The target variable is label with two classes: no and yes. The training distribution is no=23,645 and yes=7,464, giving a 3.168:1 imbalance. The data contains 50 missing feature values and three missing target labels; rows with missing labels were removed before training. There are seven numeric and eight categorical predictors, with no duplicate rows. The objective is to build a leakage-safe, reproducible pipeline and compare transparent linear and nonlinear classifiers under the same evaluation protocol.

METHODS OR PROCEDURES

All learned preprocessing is embedded inside an sklearn Pipeline and ColumnTransformer, so imputation, scaling, and encoding are refit within each training fold. Numeric columns use median imputation followed by standard scaling; categorical columns use most-frequent imputation followed by one-hot encoding with handle_unknown="ignore". All 15 original features are retained. No feature is selected using the test set, and no additional feature engineering is imposed because the data already includes the derived composite_rank field. Four classifiers are trained: DummyClassifier with a stratified strategy as the no-information baseline, LogisticRegression for an interpretable linear benchmark, RandomForest for nonlinear interactions, and GradientBoosting as a strong tabular baseline. Stratified five-fold cross-validation with shuffle=True and random_state=42 preserves the class ratio. Mean CV F1 is the primary selection metric; ROC-AUC and PR-AUC provide additional ranking evidence. The selected model is retrained on all training rows and evaluated once on the held-out test set. Balanced class weights are used for LogisticRegression and RandomForest because the minority class is about one quarter of the data. The model settings were kept deliberately modest: LogisticRegression used max_iter=1000 with class_weight=balanced; RandomForest used 200 trees with class_weight=balanced; GradientBoosting used 100 estimators; and DummyClassifier used the stratified strategy. I did not tune on the test set. The only stopping condition for the iterative logistic model was

convergence before max_iter; the tree ensembles used their fixed estimator counts. This makes the comparison reproducible, although a future version could perform a small grid search inside the training folds.

RESULTS

Test-set performance for the selected RandomForest is accuracy=0.8272, precision=0.6177, recall=0.7156, F1=0.6631, ROC-AUC=0.8887, and PR-AUC=0.7010. The CV comparison is: DummyClassifier accuracy=0.635 and F1=0.235; LogisticRegression accuracy=0.790 and F1=0.660; RandomForest accuracy=0.827 and F1=0.664; GradientBoosting accuracy=0.839 and F1=0.626. The test confusion matrix contains 8,762 true negatives, 1,403 false positives, 901 false negatives, and 2,267 true positives. The figure shows that the model identifies most minority-class cases while retaining a moderate false-positive count.

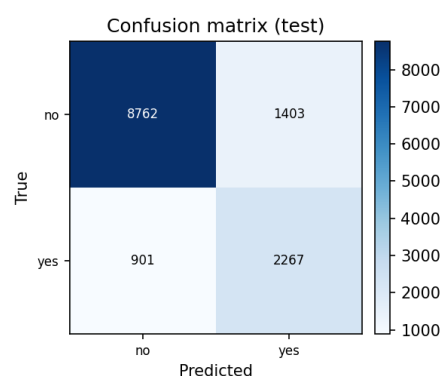


Figure 1. Test-set confusion matrix.

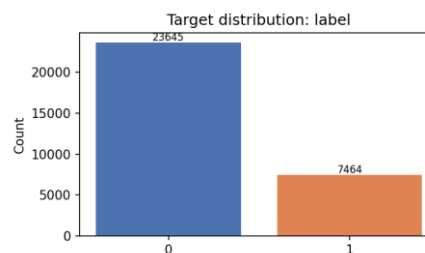


Figure 2. Target class distribution.

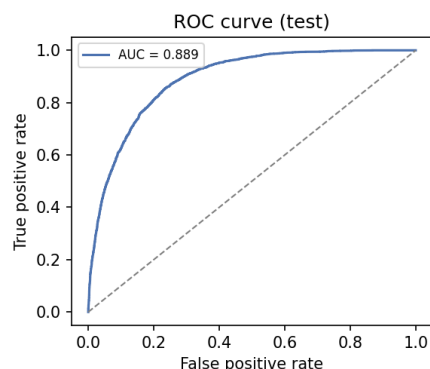


Figure 3. ROC curve for the selected model.

DISCUSSION

RandomForest is selected by mean CV F1=0.6641, but its advantage over LogisticRegression (CV F1=0.6602) is small (delta=0.0039). The two models therefore have similar overall discriminative ability, while their operating points differ. LogisticRegression has higher minority recall (0.849 versus 0.715) but lower precision (0.540 versus 0.620), so it would be preferable when missed positive cases are especially costly. RandomForest offers a more balanced precision-recall result and is the selected default under the F1 criterion. GradientBoosting obtains the highest accuracy and ROC-AUC, yet its lower recall reduces F1; this illustrates why accuracy alone would be a poor selection rule under imbalance. The test F1 of 0.6631 is close to the CV F1 of 0.6641, providing no obvious sign of overfitting. A limitation is that the hyperparameters are sensible defaults rather than tuned values; future work could evaluate threshold selection or nested tuning inside cross-validation.

CONCLUSION

The SKILL produces a reproducible classification workflow that handles missing values and mixed numeric and categorical predictors without leakage. RandomForest is the best model under the stated mean-CV-F1 criterion, although LogisticRegression is a credible alternative when recall is prioritised. The held-out results remain close to cross-validation estimates, and the confusion matrix confirms useful detection of both classes. All reported values are generated from the structured profiling and evaluation outputs.

REFERENCES

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
 - [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
 - [3] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 56-61.
- Name: LIN TONG
 Matric number: G2608825L
 Assignment: IN6227-Assignment-1, Variant-2
 Model (LLM): GPT-6
 LLM interface: Codex desktop
 GitHub: <https://github.com/tonglynn/in6227-tabular-classification-skill>